Q6. Group Anagrams

→ Basically the idea is, if two strings have the exact same characters(order not important), they are anagrams

→ In the question it's mentioned that the string only contains lowercase characters

▼ Brute Force

→ For each string, we can create a fixed size array of 26, to represent frequency of each character in the string

→ If any other string has the same representation of the array as of the string being compared with, then they are anagrams and can be grouped together

→ While iterating, if we have already checked for a string and added it to a group, we don't want to again land on those strings and begin the comparison again. That would lead to frivolous increased iterations, and faulty groups

→ So, an idea is to remove the elements if belonging to a group during the iteration itself. For that, we can not use traditional index based iteration effectively. Hence, we use an `Iterator<>` with it's `hasNext` and `next` properties for the list

→ A problem is that using the Iterator's method's we can only pass through the list once, and not loop repeatedly until we have created all the anagram groups. Unfortunately, we would only be able to create one anagram group.

→ Also, we can not use two distinct iterators simultaneusly for the same list for modification as it is not allowed and java will throw an exception.

→ So, we use two while loops. The outer loop ensures the process keeps running until the list empties completely and all anagram groups have been found. Each outer loop will initalise a new iterator instance for the current state of the list post modification. The inner loop continues iterates the current list only once, comparing all the strings with the current string of iteration, forming groups.

→ After each group is formed, we can add it into our answer data structure

▼ Slightly Better

→ The idea is to sort all the individual strings from the copy of the original array.

["act", "cat"] ⇒ Sort() ⇒ ["act", "act"]

→ By doing this, we can simply use the equality operator for two strings, as if they are anagrams and are sorted, their character sequence will also become the same.

→ We can use a HashMap to create groups using the sorted string as key, and the matching values into a list of the original(unsorted) strings

→ Lastly, we can iterate the hashmap to get individual groups, store it into the answer data structure

▼ Optimal

→ We still use a hashmap, but don’t peform sorting

→ The idea is that if two strings are anagrams, then their representive character frequency array will be exactly same, which can be used as the key for the comparison!

→ We can convert the array into a String using Java’s in built APIs, and store the converted array string as key and the original string into a list for value;

→ We can keep on checking, and populating the map based on simply the frequency character representation array of the string

→ Lastly we can simply group the individual lists together from the map and form our answer